Spatiotemporal Risk-Aware Graph Attention Network
for Financial Fraud Detection: FFD-STRA-GAT

BARSHNEYO CHAKRABORTY

20.06.2025

Motivation

- Financial fraud detection in large-scale transaction networks requires modeling:
  - Node interactions (sender–receiver relationships)
  - Temporal dynamics (evolving transaction patterns)
  - Anomaly propagation across the network
- **Key limitation of static thresholds:** a single fixed cutoff cannot distinguish a legitimate high-value transaction from a genuinely anomalous one.
- **My approach:** integrate graph-based representations with deviation-driven, learnable anomaly scoring — spatiotemporal and risk-aware.

Problem Framing

The Core Issue

Static per-account thresholds are obsolete: a customer needing to transfer Rs. 1 crore may be flagged identically to a fraudulent Rs. 1 crore transfer, if both exceed a fixed limit.

What FFD-STRA-GAT Does Differently

- Learns *context-aware tolerance* per node and edge
- Aggregates risk via graph attention, not fixed rules
- Adapts thresholds dynamically using running statistics

Graph Representation

Let $G = (V, E)$ be a directed transaction graph, where V is the vertex set and E is the edge set.

Adjacency Matrix

$$A_{ij} \in \{0, 1\}, \quad A_{ii} = \begin{cases} 1, & \text{if self-transaction exists} \\ 0, & \text{otherwise} \end{cases}$$

- A forced complete graph yields complexity: $O(n^2)$ — hypothetical, rarely occurs in practice, but included as a safety rule
- A sparse structure reduces this to: $O(kn)$

Feature Representation

17 features per transaction (sender, receiver, and edge features combined) form a 17×1 feature vector:

Node Features

$$X_i = (H_i, \text{Age}_i, \text{Loc}_i, T_{avg,i}, \epsilon_i, \text{Cat}_i)$$

$$X_j = (H_j, \text{Age}_j, \text{Loc}_j, T_{avg,j}, \epsilon_j, \text{Cat}_j)$$

Edge Features

$$e_{ij}^{(t)} = (F_{ij}^{(t)}, \Delta_{ij}^{(t)}, R_{ij}^{(t)}, \epsilon_{ij})$$

Transaction Deviation & Running Statistics

Transaction Deviation (core input to anomaly scoring)

$$\Delta_{ij}^{(t)} = |F_{ij}^{(t)} - T_{avg,ij}|$$

Average Transaction Value (temporal update)

$$T_{avg,ij}^{(t+1)} = \frac{n_{ij}^{(t)} T_{avg,ij}^{(t)} + F_{ij}^{(t+1)}}{n_{ij}^{(t)} + 1}$$

Standard Deviation (temporal update)

$$\sigma_{ij}^{(t)} = \sqrt{\frac{1}{n_{ij}^{(t)}} \sum_{t} (F_{ij}^{(t)} - T_{avg,ij}^{(t)})^2}$$

Category Mismatch & Relationship Strength

Category Mismatch

$$\text{Cat}_{mis} = \begin{cases} 1, & \text{if } \text{Cat}_i \neq \text{Cat}_j \\ \gamma, & \text{if } \text{Cat}_i = \text{Cat}_j \end{cases}$$

$\gamma = 0.5$ *fixed for now; planned to be made learnable.*

Relationship Strength (static)

$$R_{ij}^{(t)} = \frac{n_{ij}^{(t)}}{\sum_k n_{ik}}, \qquad n_{ij}^{(t+1)} = n_{ij}^{(t)} + 1$$

Tolerance Model & Unweighted Anomaly Score

Tolerance Model

$$\epsilon_{ij} = \alpha\epsilon_i + (1-\alpha)\epsilon_j + \Psi\sigma_{ij}$$

Unweighted Minimal Anomaly Score

$$AS_{ij}^{(t)} = \frac{\Delta_{ij}^{(t)} \cdot \mathrm{Cat}_{mis}}{\epsilon_{ij}}$$

This is the foundational deviation measure before learnable extensions are introduced.

Static Weight Update & Node-Level Aggregation

Temporal Weight Update (static)

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} \cdot \log\left(c + \exp\left(\frac{\Delta_{ij}^{(t)}\mathrm{Cat}_{mis}}{\epsilon_{ij}}\right)\right)(\pi + \lambda R_{ij}^{(t)})$$

Node-Level Aggregation

$$\phi_{ij}^{(t)} = w_{ij}^{(t)} \cdot AS_{ij}^{(t)}, \qquad \rho_i^{(t)} = \log\left(\sum_{j \in N(i)} \exp(\phi_{ij}^{(t)})\right)$$

Piecewise Anomaly Function & Static Threshold

Piecewise Weighted Anomaly Function

$$AS_i^{(W,t)} = \begin{cases} \rho_i^{(t)}, & \rho_i^{(t)} \leq \Theta_i^{(t)} \\ \max_j \phi_{ij}^{(t)}, & \text{otherwise} \end{cases}$$

Threshold Definition (static)

$$\Theta_i^{(t)} = Q_{1-\alpha}(\rho_i^{(t)}) + \log(|N(i)|)$$

Fraud Criterion: $AS_i^{(W,t)} > \Theta_i^{(t)} \Rightarrow \text{Fraud}$

Learnable GAT Extension: Node Transform & Attention

Node Transformation

$$h_i = W^{(t)} X_i$$

Attention Score

$$Ats_{ij}^{(t)} = \text{LeakyReLU}(a^T \tilde{z}_{ij}), \qquad \tilde{z}_{ij}^{(t)} = [W^{(t)} X_i, W^{(t)} X_j, e_{ij}^{(t)}]^T$$

where $\dim(W^{(t)} X_i) = \dim(W^{(t)} X_j) = d_h$, $\dim(e_{ij}^{(t)}) = d_e$, and a^T is the learnable attention vector.

Constructing Learnable Vectors (Xavier Initialization)

All learnable vectors $(a^T, b^T, c^T, d^T, u^T, v^T)$ follow the same construction pattern:

$$a^T = \sum_{k=1}^{d_z} a_k \delta_k^T, \qquad a_0 \in U(-\epsilon, \epsilon), \qquad \epsilon = \sqrt{\frac{6}{d_z}}$$

where $d_z = \dim(\tilde{z}_{ij}^{(t)}) = 2d_h + d_e$, and updates follow standard gradient descent:

$$a_{k+1} \leftarrow a_k - \eta \frac{\partial L}{\partial a_k}$$

Xavier/Glorot initialization used throughout for stable gradient flow.

Learnable Tolerance Updation

$$\epsilon_{ij}^{(t)} = \alpha_{ij}^{(t)}\epsilon_i + (1 - \alpha_{ij}^{(t)})\epsilon_j + \Psi_{ij}^{(t)}\sigma_{ij}$$

Three components are now learnable:

- $\Psi_{ij}^{(t)} = \text{Softplus}(d^T \tilde{z}_{ij}^{(t)})$
- $\alpha_{ij}^{(t)} = \sigma(b^T \tilde{z}_{ij}^{(t)})$

Both d^T and b^T follow the same Xavier-initialized gradient update scheme as a^T.

Learnable Unweighted Anomaly Score

Overriding the static definition:

$$AS_{ij}^{(t)} = \frac{\Delta_{ij}^{(t)} \mathrm{Cat}_{mis}}{\epsilon_{ij}^{(t)}}$$

Properties:

(a) $AS_{ij}^{(t)} \geq 0$ *(non-negativity)*

(b) $\Delta_{ij}^{(t)} \uparrow \Rightarrow AS_{ij}^{(t)} \uparrow$ *(monotonicity)*

(c) $AS_{ij}^{(t)}$ is scale invariant

$AS_{ij}^{(t)}$ is **not** *a probability — it is a normalized deviation surrogate, analogous to a z-score but with learnable tolerance replacing variance.*

Attention Normalization & Learnable Weight Update

Attention Normalization of Relationship Strength

$$R_{ij}^{(t)} = \frac{\exp(Ats_{ij}^{(t)})}{\sum_{k \in N(i)} \exp(Ats_{ik}^{(t)})}$$

Learnable Weight Update ($\pi = 1,\ c = 1$ fixed)

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} \cdot \log\left(1 + \exp\left(\frac{\Delta_{ij}^{(t)} \mathrm{Cat}_{mis}}{\epsilon_{ij}}\right)\right)(1 + \lambda_{ij}^{(t)} R_{ij}^{(t)})$$

where $\lambda_{ij}^{(t)} = \mathrm{Softplus}(c^T \tilde{z}_{ij}^{(t)})$

Learnable Weighted Anomaly Function: Why Max?

$$AS_i^{(W,t)} = \begin{cases} \rho_i^{(t)}, & \rho_i^{(t)} \leq \Theta_i^{(t)} \\ \max_j \phi_{ij}^{(t)}, & \text{otherwise} \end{cases}, \qquad \phi_{ij}^{(t)} = w_{ij}^{(t)} \cdot AS_{ij}^{(t)}$$

Why max, not mean or softmax?

Max captures worst-case risk propagation. Once the aggregate crosses the threshold, the function prioritizes the most suspicious transaction — enabling sharp localization of fraud sources rather than dilution across neighbors.

Learnable Threshold Definition

Discarding the static definition, the threshold now adapts via running statistics:

$$\Theta_i^{(t)} = \mu_i^{(t)} + k_i^{(t)} \cdot \sigma_i^{(t)} + \log(|N(i)|)$$

where $\mu_i^{(t)}, k_i^{(t)}, \sigma_i^{(t)}$ are the learnable running mean, sensitivity parameter, and standard deviation.

Running Mean Update

$$\mu_i^{(t)} = [1 - \beta_i^{(t)}]\mu_i^{(t-1)} + \beta_i^{(t)} AS_i^{(W,t)}$$

Running Variance Update

$$\sigma_i^{2,(t)} = [1 - \beta_i^{(t)}]\sigma_i^{2,(t-1)} + \beta_i^{(t)}[AS_i^{(W,t)} - \mu_i^{(t)}]^2$$

Adaptive Stability Parameter β

$$\beta_i^{(t)} = \sigma\left([u^{T,(t)} * h_i^{(t)}] + b_{\tilde{\beta}}\right)$$

Bias $b_{\tilde{\beta}}$ acts as a scalar offset controlling initial smoothing, with $\tilde{\beta}^{(0)} \in [0.02, 0.08]$:

$$b_{\tilde{\beta}}^{(t)} = \log\left(\frac{\tilde{\beta}^{(t-1)}}{1-\tilde{\beta}^{(t-1)}}\right) - u^{T,(t)} h_i^{(t)}$$

$$\tilde{\beta}^{(t)} = \sigma\left(\tilde{\beta}^{(t-1)} + \eta_{\tilde{\beta}}^{(t)}(AS_i^{(W,t)} - \mu_i^{(t)})\right)$$

$$\eta_{\tilde{\beta}}^{(t)} = \eta_0 \sigma(AS_i^{(W,t)} - \mu_i^{(t)}), \qquad \eta_0 \in [0.03, 0.07]$$

Learnable Sensitivity Parameter $k_i^{(t)}$

$$k_i^{(t)} = \text{Softplus}(v^T h_i^{(t)} + b_k^{(t)})$$

Both $u^{T,(t)}$ and v^T follow the same Xavier-initialized construction as earlier learnable vectors.

Bias update

$$b_k^{(t+1)} = b_k^{(t)} - \eta \sum_i \frac{\partial L}{\partial k_i^{(t)}} \left[\sigma(v^T h_i^{(t)} + b_k^{(t)})\right]$$

Output Layer & Loss Function

Output Layer

$$f_i^{(t)} = \frac{\Theta_i^{(t)} - AS_i^{(W,t)}}{\sqrt{\sigma_i^{(t)} + 1}}, \qquad p_i^{(t)} = \sigma(f_i^{(t)})$$

Loss Function (weighted binary cross-entropy with regularization)

$$L = -\sum_i \left[\tau y_i \log |p_i^{(t)}| + (1 - y_i) \log |1 - p_i^{(t)}| \right] + \nu \|\Omega\|^2$$

where $\tau > 1$ (class imbalance weighting) and $\nu > 0$ (L2 regularization).

Forward Propagation

1. **Node Embedding:** $h_i = W^{(t)} X_i$
2. **Attention Score:** $Ats_{ij}^{(t)} = a^T \tilde{z}_{ij}$
3. **Normalization:** $R_{ij}^{(t)} = \dfrac{\exp(Ats_{ij}^{(t)})}{\sum_{k \in N(i)} \exp(Ats_{ik}^{(t)})}$
4. **Aggregation:** $\rho_i^{(t)} = \rho_i^{(t)}(R_{ij}^{(t)}, h_j, e_{ij})$
5. **Output:** $f_i^{(t)} = \dfrac{\Theta_i^{(t)} - AS_i^{(W,t)}}{\sigma_i^{(t)} + 1}, \quad p_i^{(t)} = \sigma(f_i^{(t)})$

Backward Propagation

Gradients flow through the full computational chain via the chain rule:

$$\frac{\partial L}{\partial \Omega} = \frac{\partial L}{\partial p_i^{(t)}} \cdot \frac{\partial p_i^{(t)}}{\partial f_i^{(t)}} \cdot \frac{\partial f_i^{(t)}}{\partial AS_i^{(W,t)}} \cdot \frac{\partial AS_i^{(W,t)}}{\partial \rho_i^{(t)}} \cdot \frac{\partial \rho_i^{(t)}}{\partial R_{ij}^{(t)}} \cdot \frac{\partial R_{ij}^{(t)}}{\partial Ats_{ij}^{(t)}} \cdot \frac{\partial Ats_{ij}^{(t)}}{\partial \Omega}$$

where $\Omega = \{W^{(t)}, a^T, b^T, c^T, d^T, v^T\}$ is the full learnable parameter set.

Algorithm: FFD-STRA-GAT (Single Worker)

Require: $G = (V, E)$, X_j, X_i, e_{ij}, y_i

Ensure: $p_i^{(t)}$

Initialize $\Omega = \{W^{(t)}, a^T, b^T, c^T, d^T, v^T\}$
repeat
  for $i \in V$ **do**
    $h_i \leftarrow W^{(t)} X_i$
  end for
  for $(i, j) \in E$ **do**
    $Ats_{ij}^{(t)} \leftarrow a^T z_{ij}$
  end for
  for $i \in V$ **do**
    Compute $R_{ij}^{(t)}$
  end for
  for $i \in V$ **do**
    $\rho_i^{(t)} \leftarrow \rho_i^{(t)}(R_{ij}^{(t)}, h_j, e_{ij})$
  end for
  for $i \in V$ **do**
    Compute $AS_i^{(W,t)}$
    $f_i^{(t)} \leftarrow (\Theta_i^{(t)} - AS_i^{(W,t)})/(\sigma_i^{(t)} + 1)$
    $p_i^{(t)} \leftarrow \sigma(f_i^{(t)})$
  end for
  Compute L; $\Omega \leftarrow \Omega - \eta \nabla_\Omega L$
until convergence

Algorithm: Distributed Training via DDP

Require: Partitioned $G = (V, E)$ across P workers
Ensure: $p_i^{(t)}$
Initialize Ω
repeat
  Parallel on workers:
  for $i \in V_p$ **do**
    $h_i \leftarrow W^{(t)} X_i$
  end for
  for $(i, j) \in E_p$ **do**
    $Ats_{ij}^{(t)} \leftarrow a^T \tilde{z}_{ij}$
  end for
  Exchange boundary $Ats_{ij}^{(t)}$
  for $i \in V_p$ **do**
    Compute $R_{ij}^{(t)}, \rho_i^{(t)}$
  end for
  for $i \in V_p$ **do**
    Compute $AS_i^{(W,t)}, f_i^{(t)}, p_i^{(t)}$
  end for
  Compute L_p; All-reduce gradients
  $\Omega \leftarrow \Omega - \eta \nabla_\Omega L$
until convergence

Complexity Analysis

Let $n = |V|$, $m = |E|$, d = embedding dimension, P = workers.

Time Complexity

$$O\left(\frac{n}{P}d^2\right) + O\left(\frac{m}{P}d\right) + O\left(\frac{n+m}{P}d\right) = O\left(\frac{n+m}{P}d\right)$$

Communication Complexity

$$O(mbd) + O(|\Omega| \log P)$$

Space Complexity

$$O\left(|\Omega| + \frac{n+m}{P}d\right)$$

Mini-Batch Extension: for batch size B, k neighbors: $O(Bkd)$

Scalability Theorem

Theorem

If $m_b \ll m$, then near-linear speedup holds.

Proof Sketch

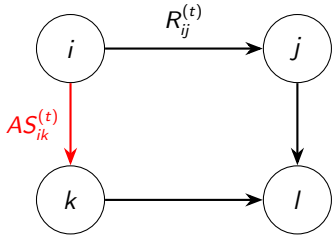
$$T_{comp} = O\left(\frac{n+m}{P} d\right), \qquad T_{comm} = O(m_b d + \|\Omega\| \log P)$$

If $T_{comm} \ll T_{comp}$:

$$T \approx O\left(\frac{n+m}{P} d\right)$$

This justifies distributed training via PyTorch DDP as the graph scales.

Illustrative Graph Representation



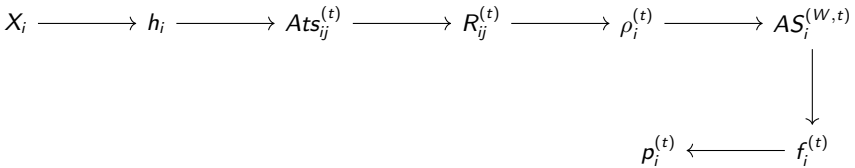
Edge weight $\propto R_{ij}^{(t)}$, anomaly $\propto AS_{ij}^{(t)}$

Distributed Architecture (DDP)

Forward: local computation + boundary exchange

Backward: gradient synchronization (All-Reduce)

End-to-End Model Pipeline



Forward propagation in FFD-STRA-GAT

Experimental Setup

Configuration

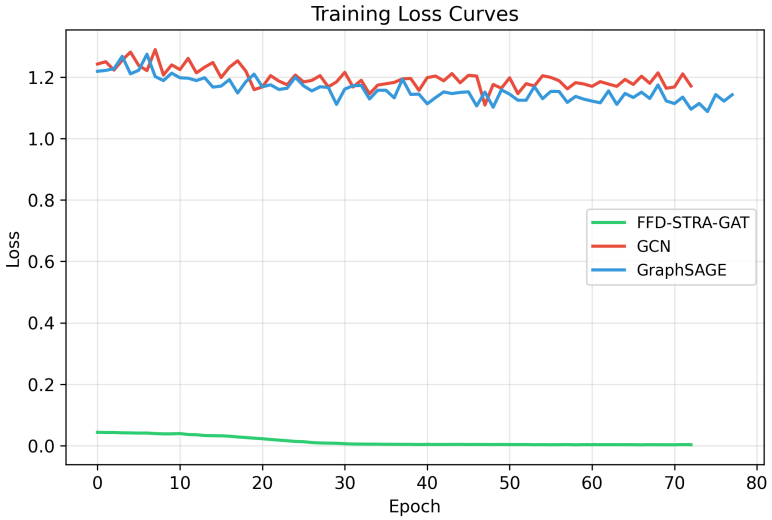
- Device: CUDA
- Epochs: 250 (early stopping)
- Hidden dim: 64
- Layers: 3
- Split: 60/20/20 (stratified)

Synthetic Dataset

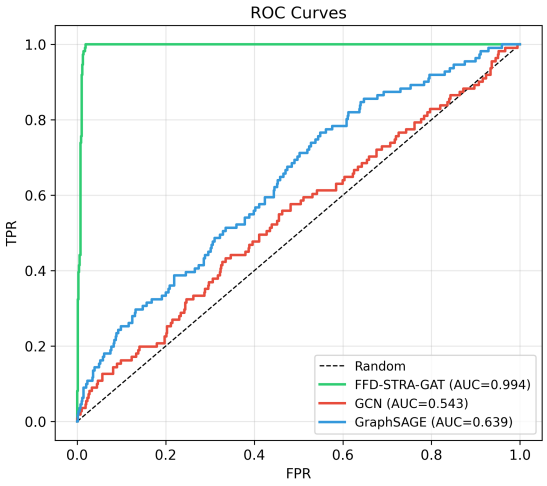
- Nodes: 800
- Edges: 5,957
- Fraud nodes: 111 (13.88%)
- Baselines: GCN, GraphSAGE

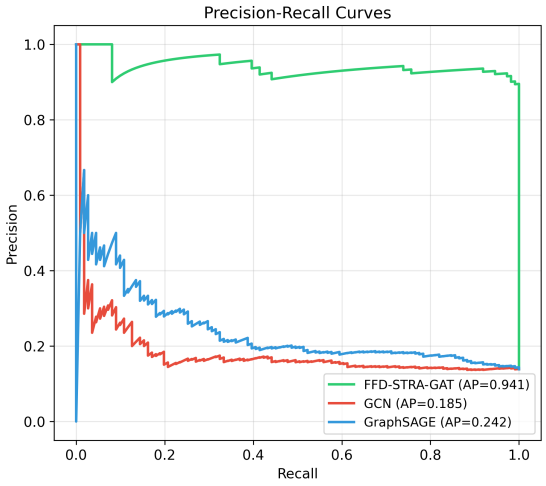
FFD-STRA-GAT converged in 73 epochs (3.58s) — best threshold $= 0.740$

Results: Training Loss Curves

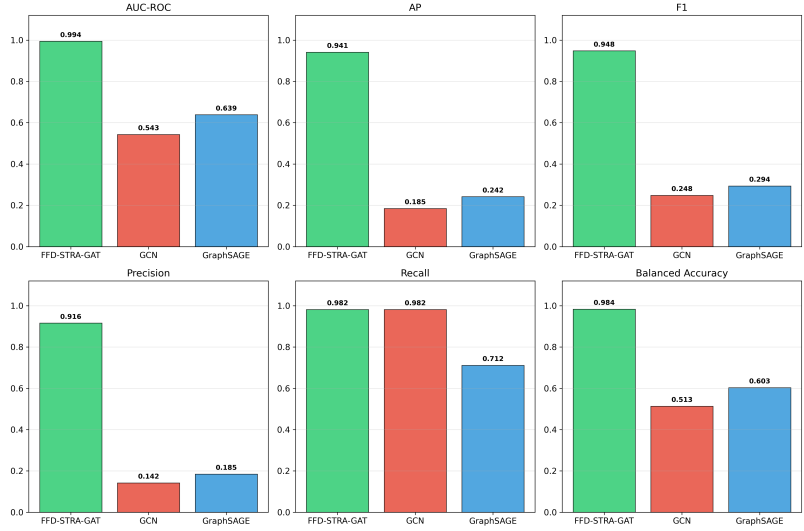


Results: ROC and Precision-Recall Curves

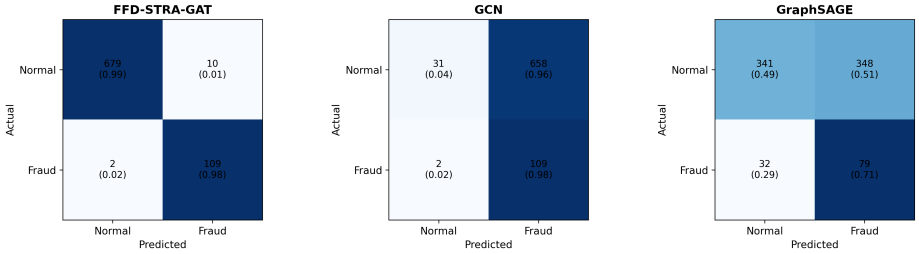




Results: Full Metric Comparison



Results: Confusion Matrices & Score Separation



Results: Summary

Model	AUC	AP	F1	Prec	Recall	BalAcc
FFD-STRA-GAT	**0.9941**	**0.9415**	**0.9478**	**0.9160**	0.9820	**0.9837**
GCN	0.5432	0.1847	0.2483	0.1421	0.9820	0.5135
GraphSAGE	0.6385	0.2418	0.2937	0.1850	0.7117	0.6033

Parameter counts: FFD-STRA-GAT — 112,431 GCN — 9,025 GraphSAGE — 17,793

Why Do GCN and GraphSAGE Underperform?

No Built-In Notion of Anomaly

GCN and GraphSAGE are generic message-passing architectures — they must learn the entire fraud concept from raw features alone, with no deviation or tolerance modeling built in.

FFD-STRA-GAT Has Fraud Signal Engineered In

$$AS_{ij}^{(t)} = \frac{\Delta_{ij}^{(t)} \mathrm{Cat}_{mis}}{\epsilon_{ij}^{(t)}}$$

acts as a mathematically principled starting point (a generalized z-score), which attention then refines — rather than discovering an equivalent signal from scratch via gradient descent.

Evidence: Loss curves show GCN/GraphSAGE plateau near $\approx$1.1–1.2 (close to random-guessing loss), while score distributions show no class separation for GCN, versus FFD-STRA-GAT's two cleanly separated peaks.

A Note on Fairness of Comparison

An Honest Caveat

This is not a strictly apples-to-apples comparison: FFD-STRA-GAT benefits from domain-specific feature engineering (the anomaly score), while GCN/GraphSAGE use only raw features.

The fairer, and more interesting, claim:

Domain-informed inductive bias — deviation-based anomaly scoring — combined with learnable graph attention substantially outperforms generic message-passing architectures that must learn fraud patterns from raw features alone.

Open question: would the gap shrink if GCN/GraphSAGE also received $AS_{ij}^{(t)}$ as an input feature? Likely yes — but learnable tolerance and attention-based aggregation should still provide an edge.

Conclusion

- FFD-STRA-GAT substantially outperforms standard GCN and GraphSAGE baselines across every metric on synthetic transaction-network benchmarks.
- Learnable, context-aware tolerance and adaptive thresholding replace brittle static rules.
- The max-aggregation design enables sharp localization of fraud sources rather than dilution.
- Distributed training via PyTorch DDP provides a theoretically justified path to scale.

Future Work

- Make γ (category mismatch), τ and ν (loss function) fully learnable.
- Validate on real-world, non-synthetic transaction datasets.
- Extend distributed training evaluation to larger worker counts.
- Explore richer edge feature sets and multi-hop propagation.